

Rapport d'Activité et d'Architecture : OCaLi

Online Cameroon Library

AZANGUE LEONEL DELMAT

26 février 2026

Résumé

Ce rapport détaille les réalisations récentes sur le projet **OCaLi** (Online Cameroon Library), décrit l'architecture des différents microservices composant le backend, et présente les ajouts et corrections apportées sur la partie frontend.

Table des matières

1	Introduction	3
2	Réalisations Récentes (Travaux Effectués)	3
3	Architecture du Backend : Les Microservices	3
4	Ajouts et Améliorations Côté Frontend	4
5	Conclusion	5

1 Introduction

L'application OCaLi est une plateforme de bibliothèque numérique reposant sur une architecture moderne. Afin de garantir une scalabilité optimale, le projet a été divisé entre un backend développé sous forme de microservices avec Node.js, Express et MongoDB, et d'un frontend utilisant notamment le framework Laravel (et ses vues Blade) pour interagir de manière transparente avec les services backend. Ce document récapitule les derniers travaux effectués sur ces deux pans matériels de l'application.

2 Réalisations Récentes (Travaux Effectués)

Au cours des derniers cycles de développement, de nombreuses corrections de bugs critiques ainsi que l'intégration de fonctionnalités majeures ont été réalisées :

- **Restauration du Flux de Paiement Nokash** : Le système de paiement par Mobile Money via la fenêtre modale (popup) Nokash a été entièrement réintégré pour les abonnements. Il a fallu corriger et ré-implémenter les scripts JavaScript ainsi que les vues de l'interface pour assurer un processus de paiement fluide.
- **Automatisation du Seeding des Données (Base de données)** : Un script de peuplement (seeding) a été implémenté pour injecter automatiquement les plans d'abonnements par défaut (dans MongoDB) et les catégories (dans MySQL) lors de l'initialisation des microservices, évitant ainsi l'absence de données sur les pages de tarification.
- **Correction de l'Authentification Sociale** : L'authentification via Google et Facebook a été réparée. Le travail a consisté à vérifier et configurer correctement les variables d'environnement OAuth requises par `docker-compose.yml` et à s'assurer du bon redémarrage du service d'authentification (*Auth Service*).
- **Résolution de l'erreur d'identifiant externe (`externalId`)** : Un problème majeur qui bloquait la mise à jour des informations de compte à cause d'un `externalId` nul a été corrigé afin d'assurer la cohérence et l'intégrité des profils utilisateurs.
- **Refonte du Lecteur PDF et Suivi de Progression** : Intégration d'un lecteur personnalisé basé sur `PDF.js` en remplacement des `iframes` natives. Cela permet un suivi précis du temps de lecture et de la page courante, tout en améliorant la fluidité.
- **Sécurisation du Contenu (Anti-Capture)** : Mise en place de mesures de protection avancées incluant un filigrane (watermark) dynamique avec l'identité de l'utilisateur, le floutage automatique lors de la perte de focus, et le blocage des raccourcis clavier de capture d'écran.

3 Architecture du Backend : Les Microservices

Le système backend repose sur une architecture en microservices orchestrée derrière une API Gateway. Voici le descriptif des fonctionnalités pour chaque composant :

1. API Gateway

- Point d'entrée unique de l'application.

- Redirige et achemine (route) de manière sécurisée toutes les requêtes entrantes vers les microservices appropriés.
- Centralise la documentation technique de l'API (Swagger UI).
- 2. Auth Service (Service d'Authentification)**
 - Chargé de la sécurité et de la connexion des utilisateurs.
 - Gère l'authentification multi-plateforme (Email/Mot de passe, Google, Facebook OAuth2).
 - Émission et validation des jetons de sécurité (JWT).
- 3. User Service (Service Utilisateurs)**
 - Gestion détaillée des profils, des préférences et des données liées aux utilisateurs.
 - Prise en compte de la mise à jour des identifiants (comme l'`externalId`).
- 4. Book Service (Service des Livres)**
 - Cœur du catalogue de la bibliothèque numérique.
 - Indexation précise des métadonnées des œuvres et liaison avec le stockage Cloud (fichiers PDF sur AWS S3 / DigitalOcean Spaces).
- 5. Subscription Service (Service d'Abonnement)**
 - Prestation relative à la facturation et aux forfaits.
 - Intégration avancée avec des passerelles de paiements locales africaines (Notch Pay, PayUnit, Nokash, PayMooney).
 - Rendu dynamique des offres de forfaits (peuplé avec les plans dans MongoDB).
- 6. Publication Service (Service des Publications)**
 - Responsable de la mise en ligne des articles de blogs, actualités internes, ou autres annonces relatives à la plateforme.
- 7. Search Service (Service de Recherche)**
 - Moteur de recherche unifié.
 - Permet de requêter de manière transversale l'ensemble du système (livres, catégories, auteurs) avec des performances optimales.

4 Ajouts et Améliorations Côté Frontend

Des travaux significatifs ont été menés pour améliorer l'expérience utilisateur et les interfaces avec le nouveau backend :

- **Intégration et Refonte des Modales de Paiement (Blade & JS) :** L'ajout majeur s'est matérialisé par la restauration de la fenêtre modale de Nokash sur l'application Laravel (FRONTEND). Le code JavaScript et les templates Blade (`NokashService.php`, etc.) ont été mis en cohérence pour capturer l'événement de paiement, afficher dynamiquement le popup à la souscription, et acquitter la transaction auprès du *Subscription Service*.
- **Refonte et Connectivité API :** Adaptation de l'interface client pour interagir nativement avec l'architecture microservices afin de récupérer les statistiques (Tableau de bord / Dashboard Stats) via de nouveaux endpoints (comme l'appel pour les abonnements, les rôles, les catalogues). Ainsi, les pages présentent dorénavant les données réelles et peuplées.

- **Améliorations Multi-Plateformes (Adaptabilité et UX) :** Plusieurs retouches ont également été faites pour la prise en charge de l’affichage offline / mobile (renommage stratégique du "Téléchargement" en "Lecture hors ligne") et l’application globale du Dark Mode, harmonisant les couleurs pour une interface moderne.
- **Gestion des Avis et Détails du Livre :** Intégration directe des descriptions, métadonnées (ISBN, Éditeur) et du système de notations/avis directement sous le lecteur, évitant les allers-retours avec les modales et enrichissant l’expérience utilisateur.

5 Conclusion

Les tâches accomplies ont grandement stabilisé l’application **OCaLi**. L’architecture microservices (Node.js/Express) fournit désormais un environnement modulable et évolutif, renforcé par un système de déploiement (Docker compose) maîtrisé. De plus, le frontend (Laravel/Vue) tire parfaitement parti des nouveaux services distribués, proposant un processus de paiement (Nokash) rétabli et une intégration sociale finalisée. L’application s’inscrit pleinement dans les critères de qualité technique et de réactivité attendus pour un service numérique de grande envergure.